Competitive Pokémon Analysis: A Graph-Based Approach to Exploring Usage and Strategies

Data Management Course Report by:
Ferrario Daniele mat. 868724
Giuseppe Maria Turco 886262
Jamil Amaiza mat. 885861

Summary

Group components and role **3**
Research questions **3**
Dataset used and Acquisition **3**
API: PokeAPI 3
Download: PokeDATA (link) 4
Data cleansing **5**
Fixing pokémon names 5
Removal of attributes from the data obtained from PokeDATA 5
Data quality **6**
Currency 6
Consistency with Naming Standards 7
Consistency in Formatting and Structure 7
Consistency Across Files and Formats 7
Completeness 7
Completeness of Pokemon_data 7
Analysis of Columns with NaN Values 8
Interpretation of the results 10
Completeness of Players 11
Analysis of columns with NaN values 11
Interpretation of the results 13
Data Model and Storage **13**
Why a graph database? 13
Why Neo4j? 13
Data Integration and Enrichment **14**
Data Schema 14
Integrity Constraints 15
Queries **16**
Query 1 16
Query 2 17
Query 3 18
Conclusions, possible improvements and future developments **19**

Group components and role

Ferrario Daniele: Data model and storage
Amaiza Jamil: Data quality
Giuseppe Maria Turco: Data acquisition and cleansing

Research questions

Profiling all pokemons and the 2024 World Championship.
Note that in the World Championship pokemon levels are standardized, so the same pokemons from different teams have the same stats.
We want to identify if there are any **recurring patterns** among the Pokémon used at the highest levels, analyzing their **stats**, **types**, and **natures**. Additionally, it may be interesting to observe how the same Pokémon are used in competitive play versus regular gameplay, focusing on differences in types.

Dataset used and Acquisition

API: PokeAPI

Notebook used: [pokeapi.ipynb]
RESTful API interface.
Provides easy access to all information regarding the game, series, manga, and events related to the Pokémon world.
The **data retrieved** from the API:

- Pokémon name
- type(s)
- statistics (hp, atk, sp. atk, def, sp. def, speed)

Implementation:

- Identification of API endpoints for the resources of interest.
- Creation of a method that takes the endpoints and, using Python's get() function, sends GET requests to the server.
- Saving the received information into a CSV file.

Dataset obtained (1302 rows x 9 columns)

name	type1	type2	hp	attack	defense	special_attack	special_defense	speed
bulbasuar	grass	poison	45	49	49	65	65	45
ivysaur	grass	poison	60	62	63	80	80	60
venosaur	grass	poison	80	82	83	10	10	80

Download: PokeDATA (link)

From the website, we were able to download the file related to the 2024 World Cup, containing JSON data regarding players, ranks, and Pokémon used with teratypes.

JSON objs obtained (693 JSON objs)


```
[{"name":
"Luca Ceribelli[IT]", "placing":1,
"record":{"wins":14,"losses":2,"ties":0},
"resistances":{"self":0.25,"opp":0.25,"oppopp":0.25},
"decklist":[
    {"id": "1008",
    "name": "Miraidon",
    "teratype": "Fairy",
    "ability": "Hadron Engine",
    "item": "Choice Specs",
    "badges":
    ["Electro Drift", "Draco Meteor", "Dazzling Gleam",
"Volt Switch"]},
    …
```

Data cleansing

Fixing pokémon names

Notebook used: [fix_names.ipynb]
There are discrepancies between the Pokémon names retrieved from the PokeAPI and those obtained from the downloaded PokeDATA file. To address this issue, a Python script (fix_names.ipynb) was created. As a first step, all names were reformatted to ensure that the first letter was lowercase, creating a consistent baseline for comparison.
In some cases, the names were adjusted dynamically by replacing spaces with dashes, for example:

- flutter-mane was Flutter Mane
- iron-hands was Iron Hands
- iron-jugulis was Iron Jugulis

However, other names had to be hardcoded due to significant differences that made dynamic changes impractical, for example:

- urshifu-rapid-strike was Urshifu [Rapid Strike Style]
- tornadus-incarnate was Tornadus [Incarnate Forme]
- sinistcha was Sinistcha [Unremarkable Form]

Removal of attributes from the data obtained from PokeDATA

Notebook used: [jsonToCSV.ipynb]
Data of no interest from the PokeDATA file were eliminated to reduce the amount of data and facilitate subsequent analysis. Additionally, the data were converted into CSV format with the following structure:

- rank
- player_name
- pokemon_name
- teratypes
- nature

This results in data redundancy but simplifies the subsequent phases of analysis and integration.

Dataset obtained (1164 rows x 5 columns)

rank	player_name	pokemon_name	teratypes	nature
1	Luca Ceribelli [IT]	Miraidon	Fairy	Hadron Engine
1	Luca Ceribelli [IT]	ogerpon-hearthflame-mask	Fire	Mold Breaker
1	Luca Ceribelli [IT]	urshifu-rapid-strike	Water	Unseen Fist
1	Luca Ceribelli [IT]	Iron-Hands	Bug	Quark Drive
1	Luca Ceribelli [IT]	Whimsicott	Dark	Prankster
1	Luca Ceribelli [IT]	Farigiraf	Water	Armor Tail
2	Yuta Ishigaki [JP]	landorus-incarnate	Ghost	Sheer Force
2	Yuta Ishigaki [JP]	urshifu-rapid-strike	Water	Unseen Fist
2	Yuta Ishigaki [JP]	Iron-Valiant	Ghost	Quark Drive
2	Yuta Ishigaki [JP]	Pelipper	Grass	Drizzle
2	Yuta Ishigaki [JP]	Amoonguss	Fire	Regenerator
2	Yuta Ishigaki [JP]	calyrex-ice	Grass	As One

Data quality

Notebook used: [completeness.ipynb]

Currency

The currency of the data provided by PokéAPI (from which the Pokémon_data dataset is sourced) depends on how frequently the API is updated to reflect the latest information from the Pokémon world.

On the other hand, the PokeDATA website (which provided the Players dataset) performs updates every 50 minutes. It fetches data from the official event locator and sends this data to Discord servers. For events, the bot takes approximately 20 minutes to read and send Discord messages, followed by a 30-minute sleep cycle after each operation. This results in an effective update interval of around 50 minutes.

Overall, while the PokeDATA dataset demonstrates strong currency with consistent updates every 50 minutes, the currency of PokéAPI data depends more on the update practices of the API maintainers.

Consistency

Notebooks used: completeness.ipynb and [fix_names.ipynb]

Consistency with Naming Standards

The Pokémon names have been standardized according to a well-defined criterion. For example, the names have been reformatted so that the first letter is always lowercase, ensuring consistency across various sources (e.g., PokeAPI and PokeDATA).

More complex differences in the names (e.g., the addition of attributes like "[Rapid Strike Style]" or "[Incarnate Forme]") have been resolved through manual adjustments, ensuring that the data aligns with competitive context and the recognized standard nomenclature.

Consistency in Formatting and Structure

Names with spaces have been dynamically transformed into names with hyphens ("-") to ensure uniformity and facilitate automated processing. The attributes for Teratypes and Natures have been aligned consistently with their respective Pokémon, eliminating potential ambiguities.

Consistency Across Files and Formats

After the data cleaning process, the dataset was exported in CSV format with a clear and well-defined structure (rank, player_name, pokemon_name, teratypes, nature). This structure ensures consistency across all rows of the dataset (1164 rows x 5 columns) and minimizes the risk of format discrepancies during data integration and analysis.

Completeness

Completeness of Pokemon_data

To evaluate the completeness of the *pokemon_data* dataset, it was decided to analyze missing values (NaN) column by column, rather than considering the entire table. This approach is particularly relevant

because, in some cases, the presence of a NaN value does not represent missing data but rather a specific characteristic of the entity being considered. For example, the column *type2* may have a NaN value for Pokémon with only one type, and this data should not be considered "missing" in the traditional sense but instead indicative of a single-type Pokémon.

Analysis of Columns with NaN Values

Using the notebook *completeness.ipynb*, the total number and percentage of NaN values were calculated for each column in the dataset. This made it possible to quickly identify columns with potential gaps and assess whether the NaN values represent an issue to be addressed or an intrinsic characteristic of the dataset.

```
import pandas as pd
import numpy as np

pokemon = pd.read_csv('poke_data.csv')
print (pokemon)
```

	name	type1	...	special_defense	speed
0	bulbasaur	grass	...	65	45
1	ivysaur	grass	...	80	60
2	venusaur	grass	...	100	80
3	charmander	fire	...	50	65
4	charmeleon	fire	...	65	80
...	...	...	...	...	...
1297	ogerpon-wellspring-mask	grass	...	96	110
1298	ogerpon-hearthflame-mask	grass	...	96	110
1299	ogerpon-cornerstone-mask	grass	...	96	110
1300	terapagos-terastal	normal	...	110	85
1301	terapagos-stellar	normal	...	110	85

```
pkmn_column_nan = pd.DataFrame(pokemon.isna().sum(),
columns=['Total NaN'])
print (pkmn_column_nan)
```

Column	Total NaN
nome	0
tipo1	0
tipo2	576
hp	0
attack	0
defense	0
special_attack	0
special_defense	0
speed	0

```
#calculation of the percentage of NaN values for each column, considering that the dataset contains 1302 rows.
pkmn_column_nan['% NaN'] = (pkmn_column_nan['Total NaN']/1302)*100
pkmn_column_nan = pkmn_column_nan.round(decimals=2)
print(pkmn_column_nan)
```

Column	Total NaN	% NaN
name	0	0.00
type1	0	0.00
type2	576	44.24.00
hp	0	0.00
attack	0	1.00
defense	0	2.00
special_attack	0	3.00
special_defense	0	4.00
speed	0	5.00

```
#completeness of the entire dataset
print(pokemon.size)
print(pokemon.isna().sum().sum())
print(pokemon.isna().sum().sum() / pokemon.size * 100)
```

11718

576
4.915514592933948

Interpretation of the results

From the outputs of the code, several important considerations about the completeness of the dataset are observed:

1. **Columns without missing values:**
   The columns name, type1, hp, attack, defense, special_attack, special_defense, and speed do not contain any NaN values, ensuring that the data for these attributes are complete for all Pokémon in the dataset.
2. **Column type2 (44.24% NaN):**
   The high percentage of missing values in this column is justified by the fact that many Pokémon have only one type. The NaN values therefore indicate the absence of a second type, which is perfectly normal for mono-type Pokémon.
3. **Overall dataset completeness:**
   > The total size of the dataset is 11,718 values (1302 rows × 9 columns).
   > The total number of missing values is 576 (all in the column type2).
   > The overall percentage of NaN values is 4.92%, which is quite low and indicates good overall completeness of the dataset.

In conclusion, most columns do not have any NaN values and ensure the availability of complete data. The only column with a significant amount of missing values is type2, but this is perfectly understandable and does not represent a gap in the dataset; instead, it reflects a natural characteristic of mono-type Pokémon. There is no need to impute or modify the NaN values in the type2 column, as they correctly represent the absence of a second type for some Pokémon.
The dataset is almost complete, with only 4.92% of NaN values, and these are concentrated in a single column (type2). No significant interventions are needed to ensure the dataset's usability.

Completeness of Players

The Players dataset contains information about the players, including their rank, name, types of Pokémon used, and their specific nature. This dataset is useful for analyzing the strategies adopted by players in a competitive context and identifying trends in Pokémon usage based on types and abilities.
To evaluate the completeness of the data, the process is carried out similarly to what was done for the Pokémon_data dataset: the missing values(NaN) are analyzed column by column.

Analysis of columns with NaN values

Using the notebook completeness.ipynb, both the total number and the percentage of NaN values present in each column of the dataset were calculated. This approach facilitated the immediate identification of columns with any missing data.

```
import pandas as pd
import numpy as np

player = pd.read_csv('Downloads/players.csv')
print (player)
```

	Rank	Player Name	Teratypes	Nature
0	1	Luca Ceribelli [IT]	Fairy	Hadron Engine
1	1	Luca Ceribelli [IT]	Fire	Mold Breaker
2	1	Luca Ceribelli [IT]	Water	Unseen Fist
3	1	Luca Ceribelli [IT]	Bug	Quark Drive
4	1	Luca Ceribelli [IT]	Dark	Prankster
...	...	...	...	...
4159	694	Gustavo Kengo [BR]	Normal	Grassy Surge
4160	694	Gustavo Kengo [BR]	Water	Armor Tail
4161	694	Gustavo Kengo [BR]	Fire	As One
4162	694	Gustavo Kengo [BR]	Stellar	Unseen Fist
4163	694	Gustavo Kengo [BR]	Fairy	Protosynthesis

```
pkmn_column_nan = pd.DataFrame(player.isna().sum(),
columns=['Total NaN'])
print(pkmn_column_nan)
```

Column	Total NaN
Rank	0
Player Name	0
Pokémon Name	0
Teratypes	0
Nature	0

```
# Calculation of the percentage of NaN values for each column,
considering that the dataset contains 4163 rows.
pkmn_column_nan['% NaN'] = (pkmn_column_nan['Total
NaN']/4163)*100
pkmn_column_nan = pkmn_column_nan.round(decimals=2)
print(pkmn_column_nan)
```

Column	Total NaN	% NaN
Rank	0	0.0
Player Name	0	0.0
Pokémon Name	0	0.0
Teratypes	0	0.0
Nature	0	0.0

```
# Completeness of the entire dataset
print(player.size)
print(player.isna().sum().sum())
print(player.isna().sum().sum() / player.size * 100)
```

20820
0
0.0

Interpretation of the results

The analysis of missing values in the Players dataset revealed that there are no NaN values in any column. This result suggests that the dataset is complete and does not have data quality issues:

> Rank: every player is correctly ranked.
> Player Name: every player has an associated name, along with their origin (indicated in square brackets, e.g., [IT] or [BR]).
> Pokémon Name, Teratypes, and Nature: all Pokémon used by players have complete information, indicating that there is no missing data regarding type choices and abilities for each Pokémon.

Since the dataset is complete and free of missing values, the percentage of NaN for all columns is 0.0%. This indicates that the dataset is ready for further analysis without the need for any cleaning processes.

Data Model and Storage

Why a graph database?

- **Natural representation of relationships:** Nodes (entities) and edges (relationships) are directly modeled, making the structure highly intuitive.
- **High performance on relational queries:** Thanks to efficient traversal along edges.
- **Scalability of connections:** As the number of connections grows, graph databases maintain consistent performance.
- **Schema flexibility:** They support dynamic schemas, allowing for the easy addition of new types of nodes or relationships.

Why Neo4j?

- **Optimization for Relationships:** Queries involving multiple hops between nodes are much faster compared to traditional relational databases.
- **Direct Access:** Relationships are stored physically, eliminating the need for costly joins.

- **Cypher:** A simple and readable language that allows you to express complex queries clearly.
- **ACID-compliant:** Ensures data consistency and reliability, even during complex operations.
- **Backup and recovery:** Tools to safeguard data in production environments.

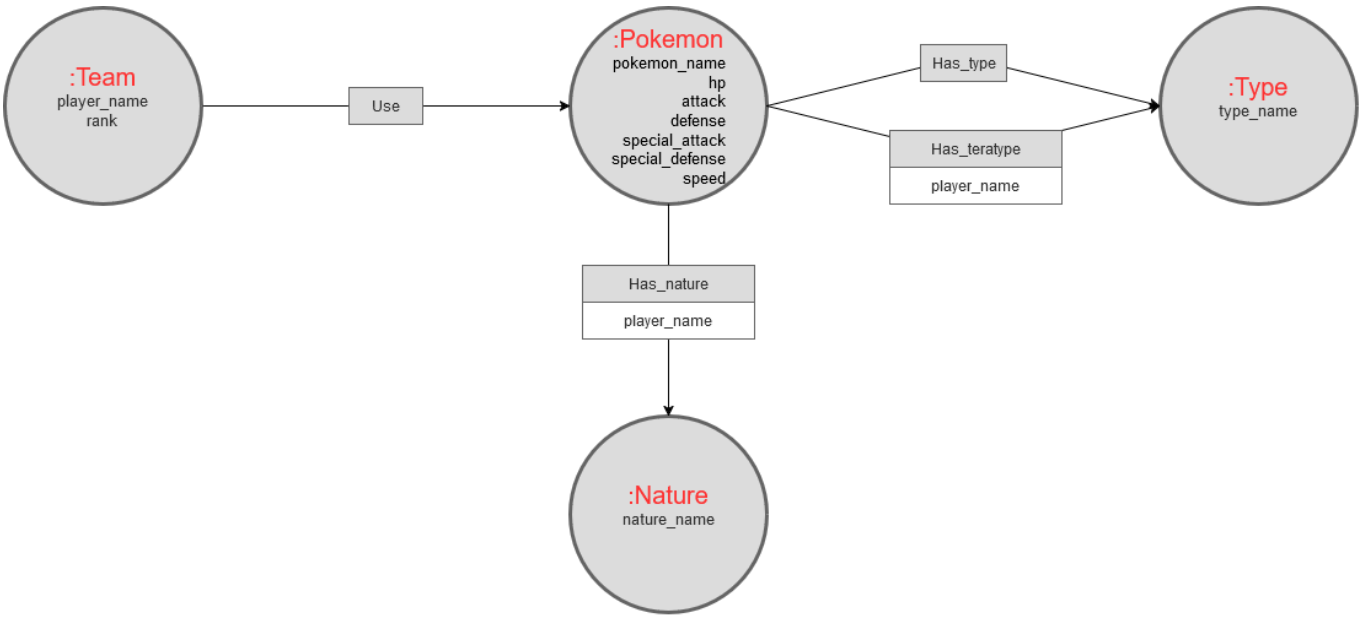
For the project, a Docker version has been used, specifically the one provided on the course's eLearning page by the professor.

Data Integration and Enrichment

Notebooks used: [nodegen3.ipynb]() and [neo4jconnect.ipynb]()

Data Schema

This modeling allows us to perform both a **general analysis** on all Pokémon and a **specific analysis** on the Pokémon that participated in the final stages of the 2024 World Championship.
Type and **Nature** have been modeled as separate nodes, as the same Pokémon in different teams can have different natures and types (teratypes). In fact, we see that the relations **Has_teratype** and **Has_nature** have the **player_name** attribute to specify the team the Pokémon belongs to. The **Has_type** relation is used to identify the Pokémon's type in the context of regular gameplay (and not competitive gameplay).



Integrity Constraints

Uniqueness constraint for the property ~_name.
Data Consistency and Quality:

- hp > 0
- attack > 0
- defense > 0
- special_attack > 0
- special_defense > 0
- speed > 0
- total > 0
- rank > 0

Queries

Notebook used: [queries.ipynb]

Query 1

Which Pokémon are the most used and with which teratype?
(Note that each Pokémon is considered only once, specifically with the Teratype corresponding to the highest number of uses)

```
MATCH (team:Team)-[:USE]->(p:Pokemon)-[r:HAS_TERATYPE]->(t:Type)
WITH p.pokemon_name AS pokemon_name,
t.type_name AS teratype,
COUNT(DISTINCT team.player_name) AS usage_count
ORDER BY usage_count DESC
WITH pokemon_name,
COLLECT({teratype: teratype, usage_count: usage_count}) AS teratypes
RETURN pokemon_name,
       teratypes[0].teratype AS most_used_teratype,
       teratypes[0].usage_count AS max_usage_count
ORDER BY max_usage_count DESC
LIMIT 10;
```

pokemon_name	most_used_teratype	max_usage_count
rillaboom	fire	195
raging-bolt	electric	160
incineroar	ghost	150
urshifu-rapid-strike	water	134
ogerpon-hearthflame	fire	110
miraidon	fairy	104
ogerpon-cornerstone	rock	103
farigiraf	water	102
iron-hands	grass	84
calyrex-ice	fire	83

Query 2

Difference in stats between competitive and regular gameplay pokemons.

(Note that "stats" refers to the sum of their individual statistics, and that each Pokémon should be considered only once, even if it is used with multiple Teratypes)

```
MATCH (p:Pokemon)-[r:HAS_TERATYPE]->(t:Type)
WITH ROUND(AVG(p.hp + p.attack + p.defense + p.special_attack + p.special_defense +
 p.speed), 2) AS avg_competitive_stats


MATCH (p:Pokemon)
WITH ROUND(AVG(p.hp + p.attack + p.defense + p.special_attack + p.special_defense +
p.speed), 2) AS avg_regular_stats, avg_competitive_stats


// Calcolare la differenza nelle statistiche medie
RETURN avg_regular_stats, avg_competitive_stats,
ROUND(avg_regular_stats - avg_competitive_stats, 2) AS stats_difference;
```

avg_regular_stats	avg_competitive_stats	stats_difference
445.86	553.43	-107.57

Query 3

Differences in type/dual type between Pokémon used in competitive play and in regular gameplay

(Note that the pokemon used for the comparison are the one from the first query)

```
MATCH (team:Team)-[:USE]->(p:Pokemon)-[r:HAS_TERATYPE]->(t:Type)
WITH p.pokemon_name AS pokemon_name,
t.type_name AS teratype,
COUNT(DISTINCT r.player_name) AS usage_count
ORDER BY pokemon_name, usage_count DESC
WITH pokemon_name,
COLLECT({teratype: teratype, usage_count: usage_count}) AS teratype_usage
MATCH (p:Pokemon {pokemon_name: pokemon_name})-[:HAS_TYPE]->(type:Type)
WITH pokemon_name, teratype_usage[0].teratype AS most_used_teratype,
teratype_usage[0].usage_count AS usage_count,
COLLECT(type.type_name) AS types
RETURN pokemon_name, most_used_teratype, types
ORDER BY usage_count DESC
LIMIT 10;
```

pokemon_name	most_used_teratype	types
rillaboom	fire	grass
raging-bolt	electric	electric, dragon
incineroar	ghost	fire, dark
urshifu-rapid-strike	water	fighting, water
ogerpon-hearthflame-mask	fire	grass, fire
miraidon	fairy	electric, dragon
ogerpon-cornerstone-mask	rock	grass, rock
farigiraf	water	normal, psychic
iron-hands	grass	fighting, electric
calyrex-ice	fire	psychic, ice

Conclusions, possible improvements and future developments

Query 1 provides an overview of the most used Pokémon, helping new players understand which Pokémon to focus on if they want to improve. Queries 2 and 3, on the other hand, highlight the differences between the competitive scene and regular gameplay.

Specifically, Query 2 shows how, in competitive play, Pokémon with the highest stats are mostly used, while Query 3 demonstrates that competitive players tend to use Pokémon in ways that differ from their intended roles.

As for future developments, it could be interesting to analyze the items and moves used by Pokémon in competitive play, or to conduct a more comprehensive analysis of teams by examining how frequently Pokémon appear together with others, considering teratypes, moves, and items as well.